

Bluesky iPhone Posts

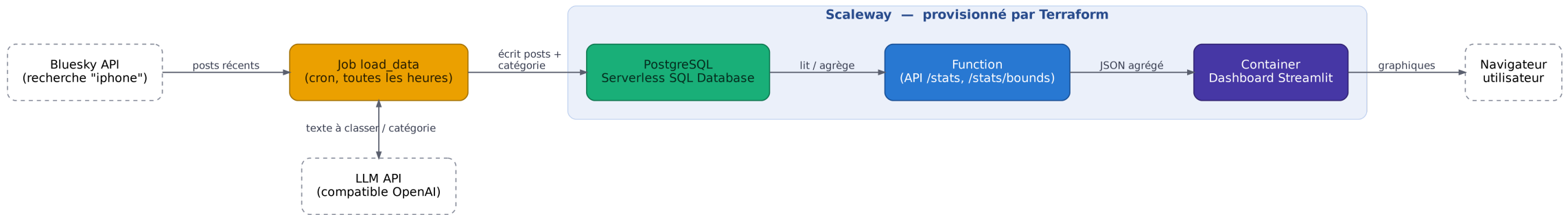
Pipeline serverless & dashboard — Terraform + Scaleway

Observer en temps quasi-réel ce qui se dit sur l'iPhone sur Bluesky, sans gérer le moindre serveur.

Le contexte

- Bluesky expose une API de recherche publique — on peut y chercher tous les posts contenant un mot-clé.
- **Idée** : suivre en continu les posts mentionnant "*iphone*", les classer par thème et visualiser les tendances.
- **Contrainte** : tout doit tourner en **serverless** — pas de VM à gérer, coût quasi nul à l'arrêt.
- **Résultat** : un pipeline de collecte → classification IA → stockage → API → dashboard, entièrement défini en Terraform.

Vue d'ensemble



Quatre briques, toutes provisionnées par Terraform et hébergées sur **Scaleway**.

Les 4 briques du pipeline

1. `code/load_data`

Job cron horaire : recherche Bluesky → classification LLM → écriture en base.

2. `terraform/db.tf`

Base PostgreSQL Serverless, seule source de vérité.

3. `code/function`

Function serverless, seule à parler à la base, expose une API JSON déjà agrégée.

4. `code/dashboard`

Container Streamlit, consomme l'API et affiche les graphiques.

1 · Collecte & classification

`code/load_data/main.py` — exécuté comme **Job Scaleway**, cron `0 * * * *` (toutes les heures)

- Connexion à Bluesky (`atproto`) avec un compte applicatif dédié.
- Recherche jusqu'à **500 posts récents** contenant *"iphone"* (dernière heure).
- Chaque texte est envoyé à un **LLM** (`groq/openai/gpt-oss-20b` , endpoint compatible OpenAI) avec 10 catégories candidates : rumeur produit, publicité, plainte, anecdote, humour, comparatif, bug technique, neutre, hors-sujet...
- Le post (auteur, texte, catégorie, date) est stocké en PostgreSQL via **Peewee**.
- Une erreur de classification sur un post n'interrompt pas les suivants.

2 · Stockage

`terraform/db.tf` — **Scaleway Serverless SQL Database** (PostgreSQL)

- Autoscaling **0** → **8 CPU** : rien ne tourne (donc ne coûte rien) en dehors des accès.
- Un compte IAM applicatif dédié (`db_app`) avec une policy `ServerlessSQLDatabaseReadWrite` .
- Sa clé API sert à construire l'URL de connexion, réutilisée :
 - par la **Function**, pour la lecture,
 - par le **Job**, pour l'écriture (injectée comme secret Scaleway).
- Une seule table : `Post(id, author_handle, text, category, posted_at)` .

3 · API d'agrégation

`code/function/function/handler.py` — **Scaleway Serverless Function**, seule à détenir l'accès à la base

Route	Rôle
GET /	healthcheck
GET /stats/bounds	dates min/max des posts en base
GET /stats?since&until	comptage par catégorie, répartition par longueur, top emojis

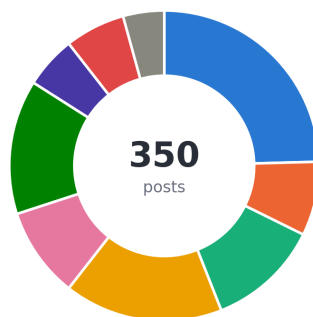
Tout le calcul lourd (parcours des lignes, comptage, regex emojis) se fait **côté serveur** : le dashboard ne reçoit jamais les posts bruts, seulement du JSON déjà agrégé.

4 · Dashboard

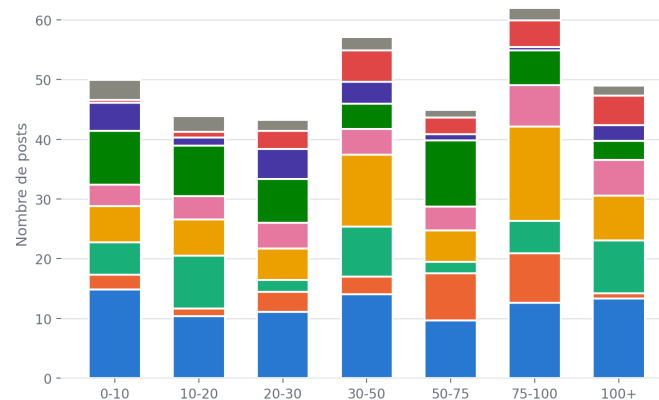
`code/dashboard/app.py` — **Streamlit**, déployé comme **Scaleway Container** (scale 0 → 1)

- Slider de période (`/stats/bounds`) puis appel `/stats` (cache 60 s).
- Camembert des catégories, histogramme empilé par longueur, emojis préférés — 8 catégories gardent une couleur fixe, le reste est replié sous *"autre"*.

Types de posts sur la période



Types de posts par longueur (mots)



■ rumeur produit ■ plainte ■ photo / création ■ comparatif ■ autre
■ publicité ■ anecdote perso ■ humour ■ bug technique

Infrastructure as Code

Tout le déploiement est décrit dans `terraform/`, provider officiel `scaleway/scaleway` :

Fichier	Rôle
<code>db.tf</code>	Provider, base PostgreSQL, IAM de connexion
<code>registry.tf</code>	Registre de containers privé (images Job + Dashboard)
<code>function.tf</code>	Build des dépendances Python, zip, déploiement Function
<code>dashboard.tf</code>	Build & push image Docker, déploiement Container
<code>job.tf</code>	Build & push image Docker, Job + cron horaire + secrets
<code>secrets.tf</code>	Secrets Scaleway référencés par le Job
<code>outputs.tf</code>	URL Function, URL Dashboard, connection string

Auto-rebuild à chaque changement

- Chaque brique buildable (Function, Job, Dashboard) calcule un **hash SHA-256** du contenu de son dossier de code.
- Ce hash sert de `trigger` à un `null_resource` (ou d'input à `archive_file` pour la Function).
- Résultat : modifier un fichier source suffit — un simple `terraform apply` reconstruit et republie l'image ou le zip concerné.
- Pas de pipeline CI/CD externe nécessaire pour un projet de cette taille.

Développement local

- **Function** : `code/function/local_server.py` — serveur HTTP stdlib qui simule les events Scaleway, pour tester `handler.py` sans déployer.
- **Function (packaging manuel)** : `code/function/package.sh` — reconstruit `function.zip` avec les dépendances compilées dans l'image Docker officielle du runtime, déploie via `scw function deploy`.
- **Dashboard / Job** : chacun a son `Dockerfile`, buildable indépendamment (`docker build code/dashboard`, `docker build code/load_data`).

Stack technique

Infra & compute

- Terraform (`scaleway/scaleway`)
- Scaleway Serverless Functions
- Scaleway Containers
- Scaleway Serverless Jobs (cron)

Données

- Scaleway Serverless SQL Database (PostgreSQL)
- ORM Peewee

Intelligence

- LLM via API compatible OpenAI (SDK `openai`)

Source & visualisation

- API Bluesky (`atproto`)
- Streamlit + Plotly

Merci

Code, Terraform et README complet : dépôt `proj - terraform`